

Intro to AI Agents using smolagents

Sergio Paniego Blanco (@sergiopaniego)

Machine Learning Engineer @ Hugging Face



Hi, I'm Sergio 🙌

- Machine Learning Engineer @ Hugging Face
 - Multimodality, Post-Training, Agents...
- PhD in AI
- Open source and AI → GSoC / ML Circle Madrid
- Teaching and Sharing



What is an AI Agent?

System that leverages an AI model to interact with its environment in order to achieve a user-defined objective. It combines reasoning, planning, and the execution of actions (often via external tools) to fulfill tasks



AI components

- Brain (AI model) 
- Body (capabilities and tools) 

AI models: LLM (text-to-text) or VLM (image-text-to-text).

- GPT 5 from OpenAI, Qwen models, Llama from Meta, Gemini from Google ...

How do they take actions? → they have access to tools!

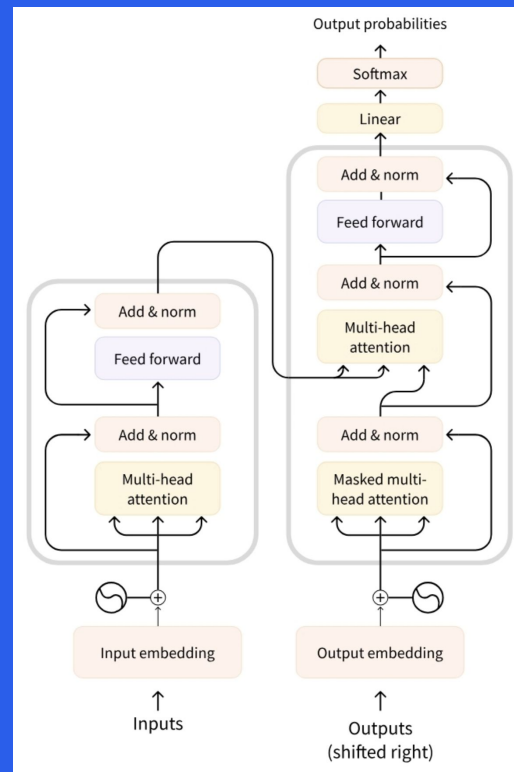
Levels of Agency

Agency level	Description	Short name	Example code
	LLM output has no impact on program flow	Simple processor	<code>process_llm_output(llm_response)</code>
★	LLM output controls an if/else switch	Router	<code>If llm_decision(): path_a() else: path_b()</code>
★★	LLM output controls function execution	Tool call	<code>run_function(llm_chosen_tool, llm_chosen_args)</code>
★★★	LLM output controls iteration and program continuation	Multi-step Agent	<code>while llm_should_continue(): execute_next_step()</code>
◇★★★	One agentic workflow can start another agentic workflow	Multi-Agent	<code>If llm_trigger(): execute_agent()</code> ◇
★★★★	LLM acts in code, can define its own tools/start other agents	Code Agents	<code>def custom_tool(args): ...</code>

What is an LLM?

AI system that excels at understanding and generating human language.

- Trained on vast amounts of text data (multimodal too)
- They use transformers 
 - Encoders
 - **Decoders**
 - Seq2Seq (encoder-decoder)



Types of transformers

Encoders:

- Example: ModernBERT, BERT
- Use case: text classification, semantic search, NER
- Size: millions of params

Decoders:

- Example: Llama, Gemini, GPT...
- Use case: text generation, chatbots, code generation.
- Size: billions of params

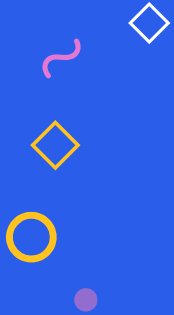
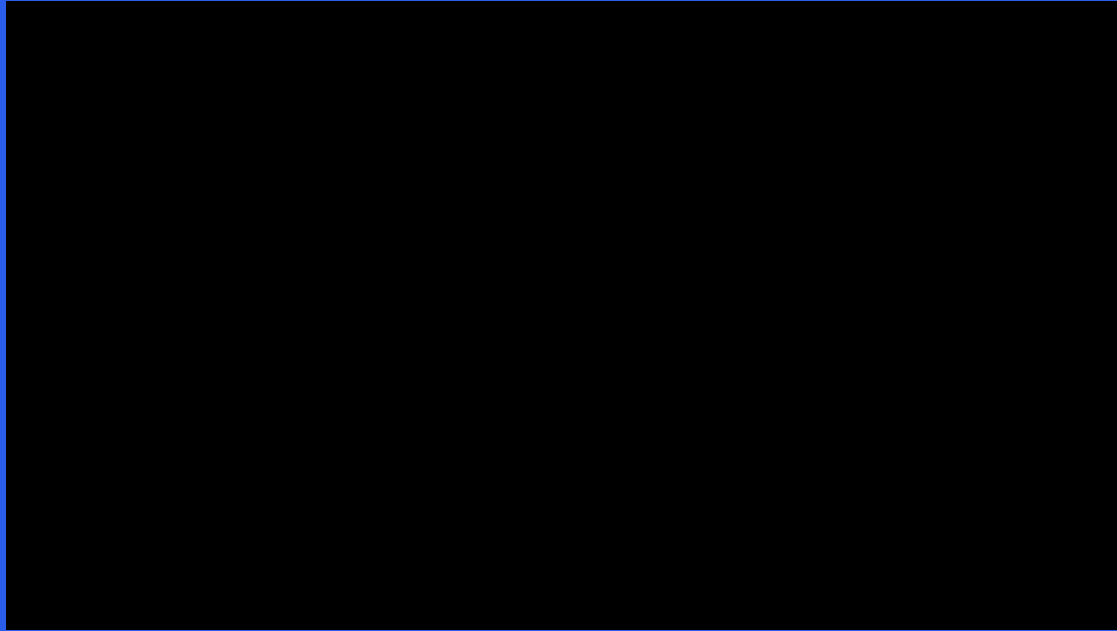
Seq2Seq (encoder-decoder):

- ◆ Example: T5, BART
 - Use case: translation, summarization
 - Size: millions of params

Typical LLM systems are decoders: DeepSeek, GPT, Gemini, Llama, Gemma, SmoLLM...

Goal of LLMs

Predict the next token given a sequence of previous tokens
(autoregressive)



Base vs Instruct models

Base model: trained on raw data to predict the next token

Instruct model: fine-tuned to follow instructions and engage in conversations. Use chat templates

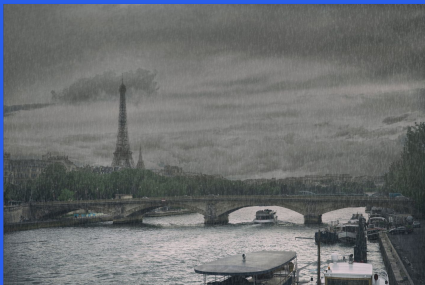
```
messages = [  
  {"role": "system", "content": "You are a helpful assistant focused on technical topics."},  
  {"role": "user", "content": "Can you explain what a chat template is?"},  
  {"role": "assistant", "content": "A chat template structures conversations between users and AI models..."},  
  {"role": "user", "content": "How do I use it?"},  
]
```

What are tools?

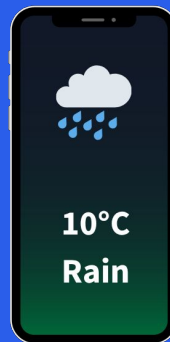
Function given to the LLM with a clear goal

- Web search, image generation, retrieval, API interface...

What's the current
weather in Paris?



The LLM
hallucinates



Agent that has
WebSearch Tool

What are tools?

Function given to the LLM with a clear goal

- Web search, image generation, retrieval, API interface...

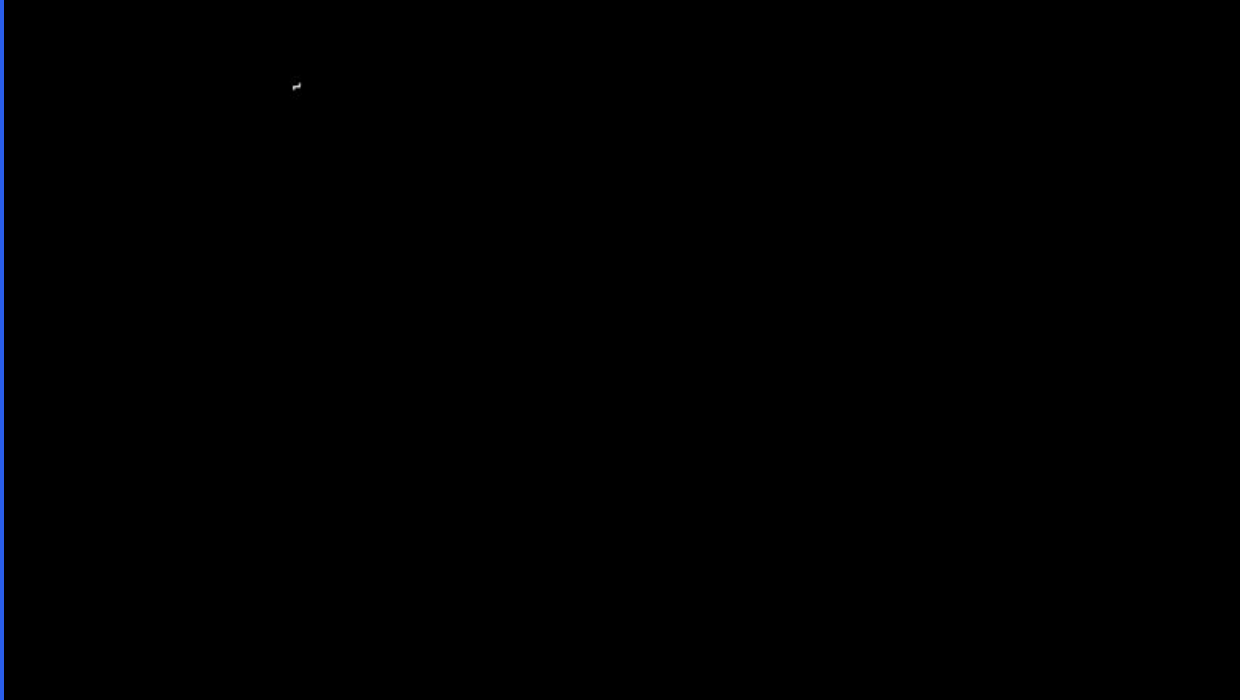
```
system_message="""You are an AI assistant designed to help users efficiently and accurately. Your primary goal is to provide helpful, precise, and clear responses.
```

```
You have access to the following tools:
```

```
Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int  
"""
```

AI Agent workflow

Thinking (Thought) → Acting (Act) and observing (Observe)



AI Agent workflow

Thinking (Thought) → Acting (Act) and observing (Observe)

```
system_message="""You are an AI assistant designed to help users efficiently and accurately. Your primary goal is to provide helpful, precise, and clear responses.
```

```
You have access to the following tools:
```

```
Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int
```

```
You should think step by step in order to fulfill the objective with a reasoning divided into Thought/Action/Observation steps that can be repeated multiple times if needed.
```

```
You should first reflect on the current situation using `Thought: {your_thoughts}`, then (if necessary), call a tool with the proper JSON formatting `Action: {JSON_BLOB}`, or print your final answer starting with the prefix `Final Answer:`
```

```
"""
```

Thought: Internal Reasoning & ReAct

- Thoughts represent the Agent's internal reasoning and planning processes to solve the task
- **ReAct approach:** “Reasoning” (Think) + “Acting” (Act). Prompting technique that appends “Let’s think step by step” before letting the LLM decode the next tokens.

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) *There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓*

Actions: Engaging with environment

- Actions are concrete steps an AI agent takes to interact with its environment.
- Types of Agent Actions:
 - **JSON Agent:** action is specified in JSON format.
 - **Code Agent:** agent writes code block that is interpreted externally.

```
Thought: I need to check the current weather for New York
Action:
{
  "action": "get_weather",
  "action_input": {"location", "New York"}
}
```

Actions: Engaging with environment

Instruction: Determine the most cost-effective country to purchase the smartphone model "CodeAct 1". The countries to consider are the USA, Japan, Germany, and India.

Available APIs

[1] lookup_rates(country: str) -> (float, float)
[2] convert_and_tax(price: float, exchange_rate: float, tax_rate: float) -> float

[3] estimate_final_price(converted_price: float, shipping_cost: float) -> float
[4] lookup_phone_price(model: str, country: str) -> float
[5] estimate_shipping_cost(destination_country: str) -> float

LLM Agent using [Text/JSON] as Action

Think I should calculate the phone price in USD for each country, then find the most cost-effective country.

Action Text: lookup_rates, Germany
JSON: {"tool": "lookup_rates", "country": "Germany"}

Environment 1.1, 0.19

Action Text: lookup_phone_price, CodeAct 1, Germany
JSON: {"tool": "lookup_phone_price", "model": "CodeAct 1", "country": "Germany"}

Environment 700

Action Text: convert_and_tax, 700, 1.1, 0.19
JSON: {"tool": "convert_and_tax", "price": 700, "exchange_rate": 1.1, "tax_rate": 0.19}

Environment 916.3

[... interactions omitted (look up shipping cost and calculate final price) ...]

Action Text: lookup_rates, Japan
JSON: {"tool": "lookup_rates", "country": "Japan"}

[... interactions omitted (calculate final price for all other countries) ...]

Response The most cost-effective country to purchase the smartphone model is Japan with price 904.00 in USD.

Fewer Actions Required!

CodeAct: LLM Agent using [Code] as Action

Think I should calculate the phone price in USD for each country, then find the most cost-effective country.

Action

```
countries = ['USA', 'Japan', 'Germany', 'India']
final_prices = {}

for country in countries:
    exchange_rate, tax_rate = lookup_rates(country)
    local_price = lookup_phone_price("xAct 1", country)
    converted_price = convert_and_tax(
        local_price, exchange_rate, tax_rate
    )
    shipping_cost = estimate_shipping_cost(country)
    final_price = estimate_final_price(converted_price, shipping_cost)
    final_prices[country] = final_price

most_cost_effective_country = min(final_prices, key=final_prices.get)
most_cost_effective_price = final_prices[most_cost_effective_country]
print(most_cost_effective_country, most_cost_effective_price)
```

Environment 1.1, 0.19

Response The most cost-effective country to purchase the smartphone model is Japan with price 904.00 in USD.

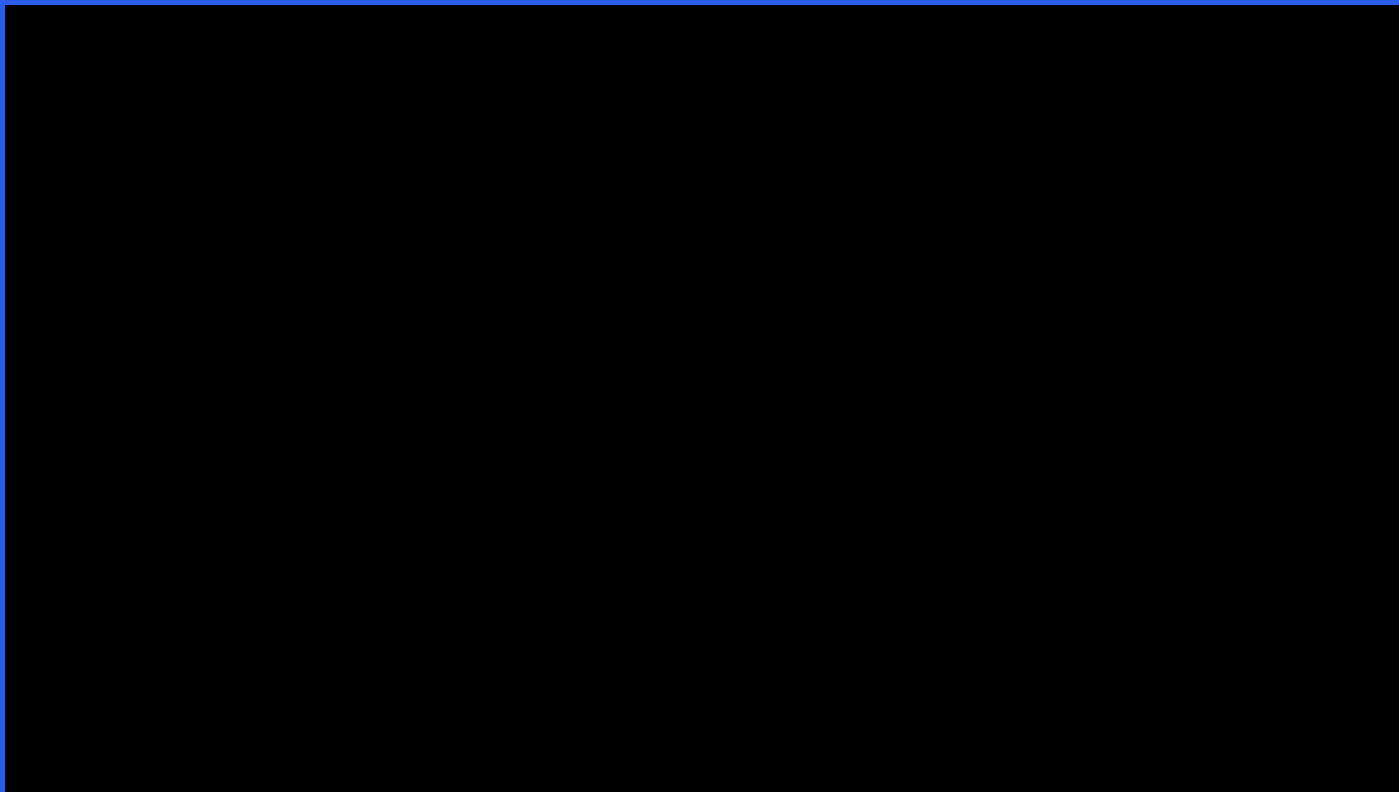
Control & Data Flow of Code Simplifies Complex Operations

Re-use 'min' Function from Existing Software Infrastructures (Python library)

Observe: Integrating Feedback to Reflect and Adapt

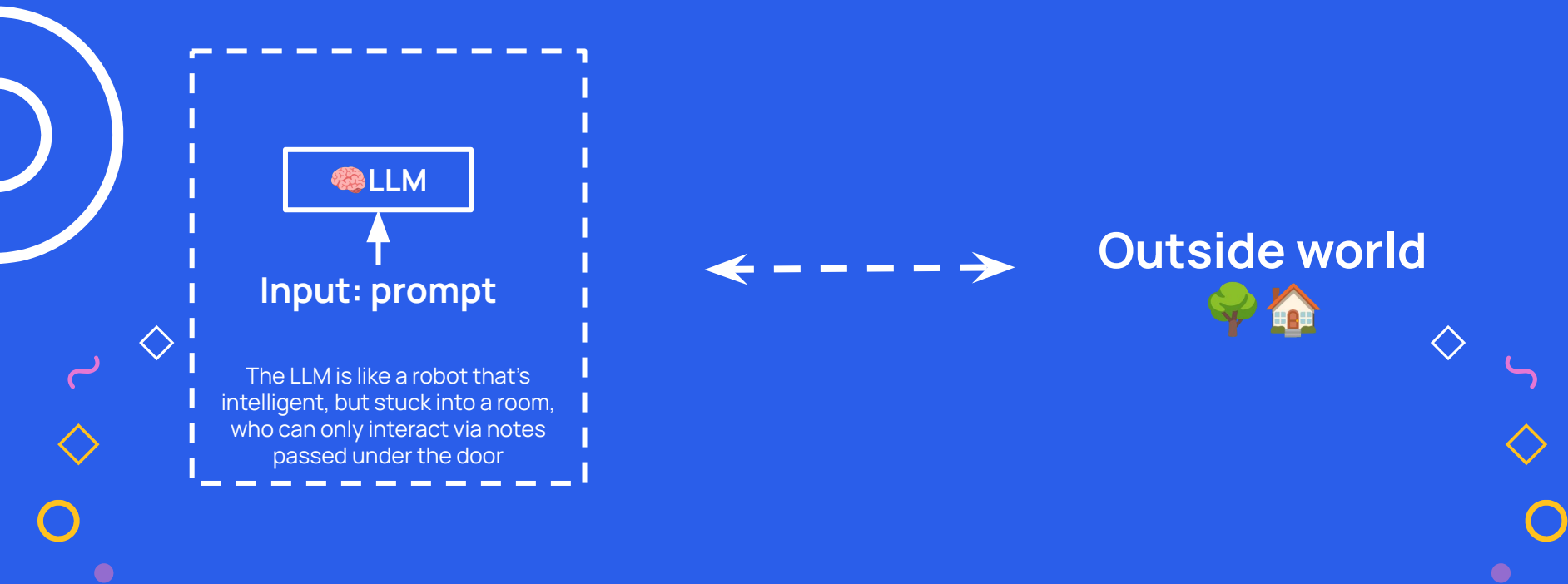
- How an agent perceives the consequences of its actions.
- Observation phase:
 - Agent collects feedback
 - Appends results
 - Adapts its strategy
- How are results appended?
 - Parse the action
 - Execute the action
 - Append the result as an observation

ReAct agent



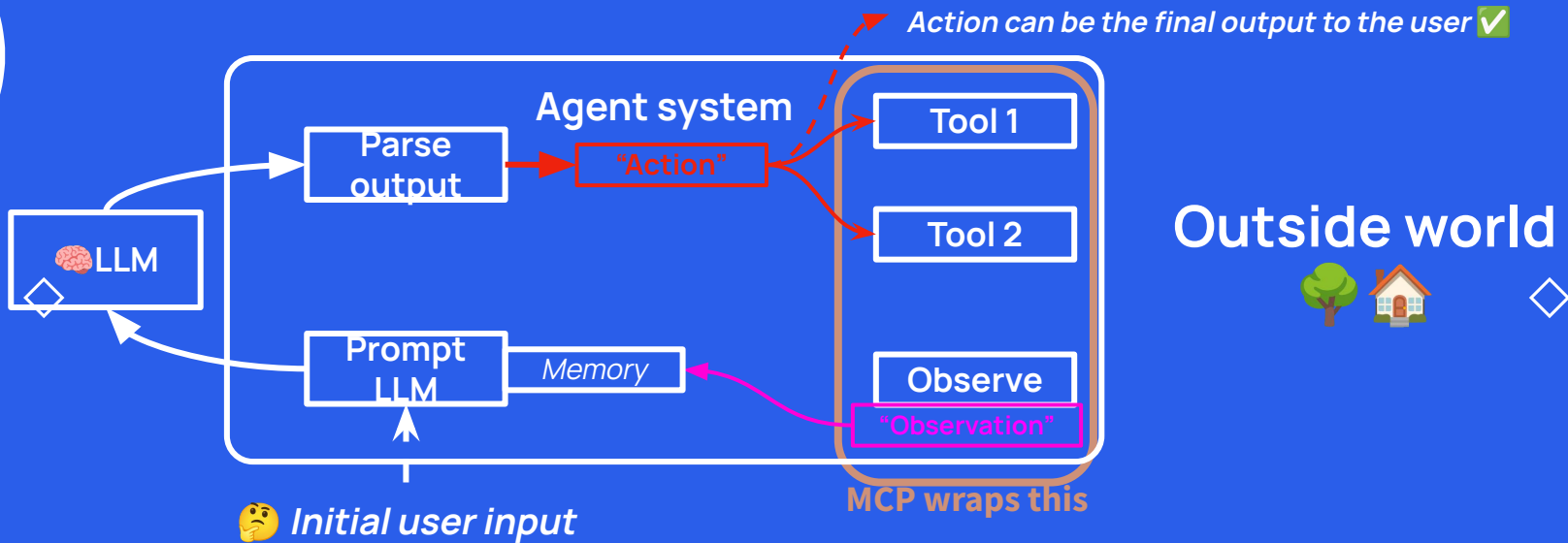
Why an agent?

Need to de-isolate an LLM from the world.



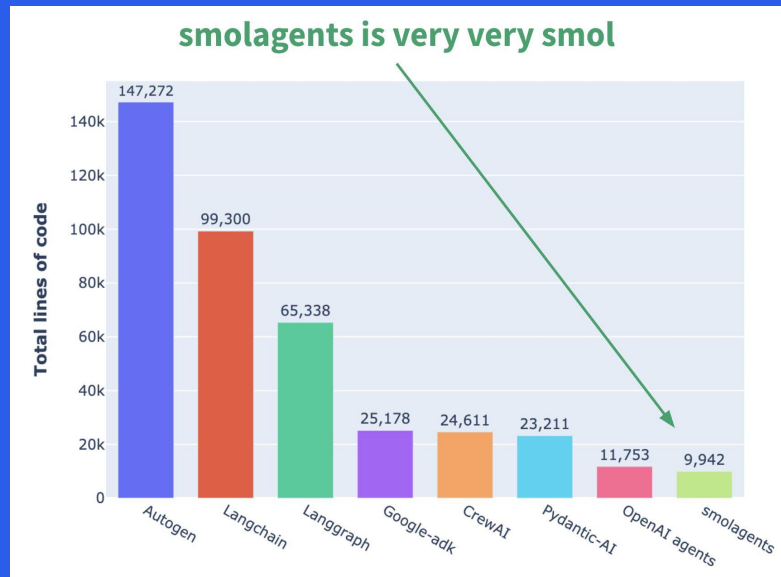
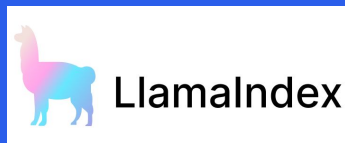
What is MCP?

Protocol that proposes and easy wrapper on tools (and prompts)



Agentic Frameworks

smolagents (Hugging Face)
LangGraph (LangChain)
Llama-Index
OpenAI Agents SDK (OpenAI)
Autogen by Microsoft



Why smolagents

- **smolagents** is a simple yet powerful framework for building AI agents.
- **Advantages:**
 - Simplicity
 - Flexible LLM Support (transformers, HfApiModel, vLLM, OpenAI...)
 - Code-First Approach
 - HF Hub integration



Why smolagents

In smolagents, agents operate as multi-step agents:

- One thought
- One tool call and execution

Type of agents:

- CodeAgent! 
- ToolCallingAgent 



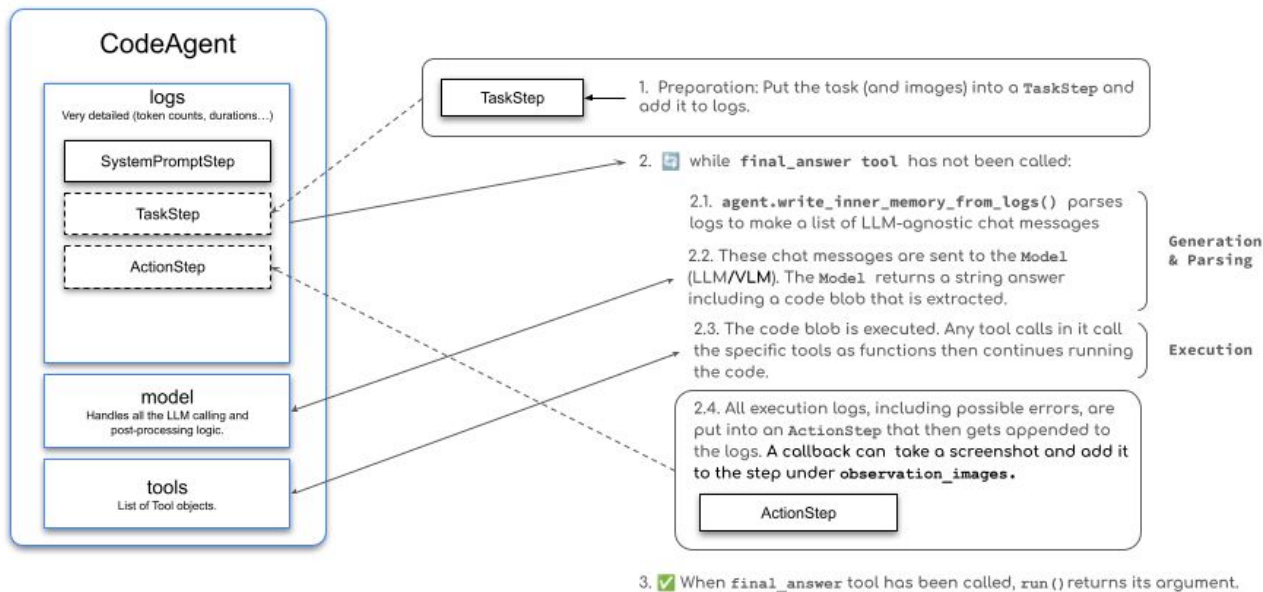
Building Agents that use code

Why Code Agents?

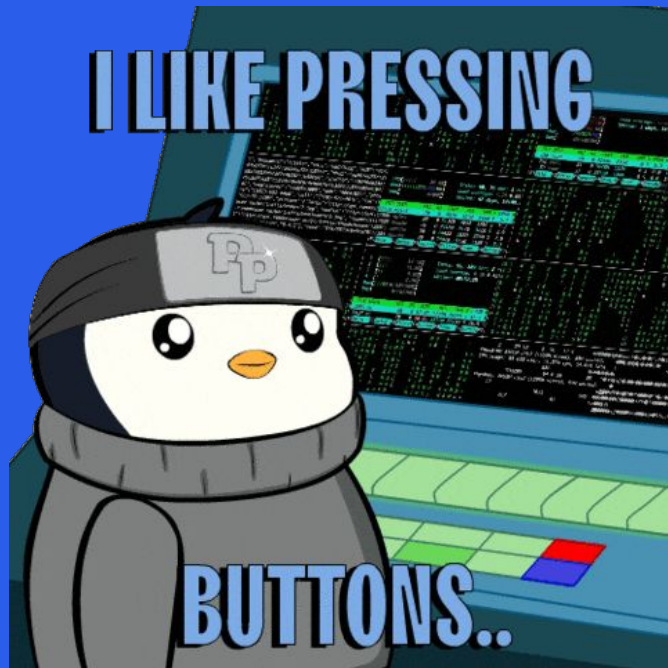
- **Composability:** easily combine and reuse actions
- **Object Management:** work directly with complex structures like images
- **Generality:** express any computationally possible task
- **Natural for LLMs:** High-quality code is already present in LLM training data

Building Agents that use code

How `CodeAgent.run()` works



Building Agents that use code



Notebook

Writing actions as code snippets

Use built-in tool-calling capabilities of LLM providers to generate tool calls as JSON structures

```
for query in [  
    "Best catering services in Gotham City",  
    "Party theme ideas for superheroes"  
]:  
    print(web_search(f"Search for:{query}"))
```

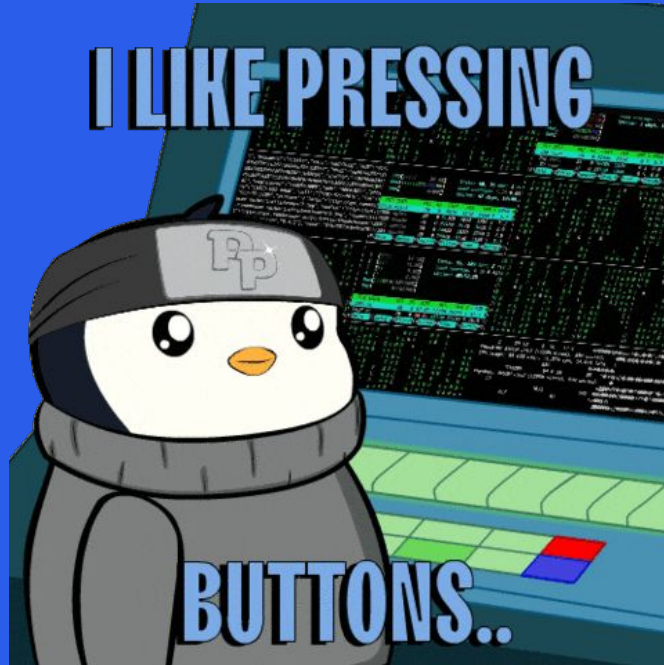
Tools

- The LLM need an interface description with:
 - **Name:** web_search
 - **Tool description:** searches the web for specific queries
 - **Input types and descriptions:** query (string)
 - **Output types:** string containing the search results.
- How are they created:
 - @tool decorator
 - Subclass of Tool

Tools

- smolagents also has a default toolbox:
 - PythonInterpreterTool
 - FinalAnswerTool
 - UserInputTool
 - DuckDuckGoSearchTool
 - GoogleSearchTool
 - VisitWebpageTool
- Tools can be shared and imported:
 - To/from HF Hub
 - From HF Spaces
 - From LangChain Tools
 - From MCP Servers

Tools

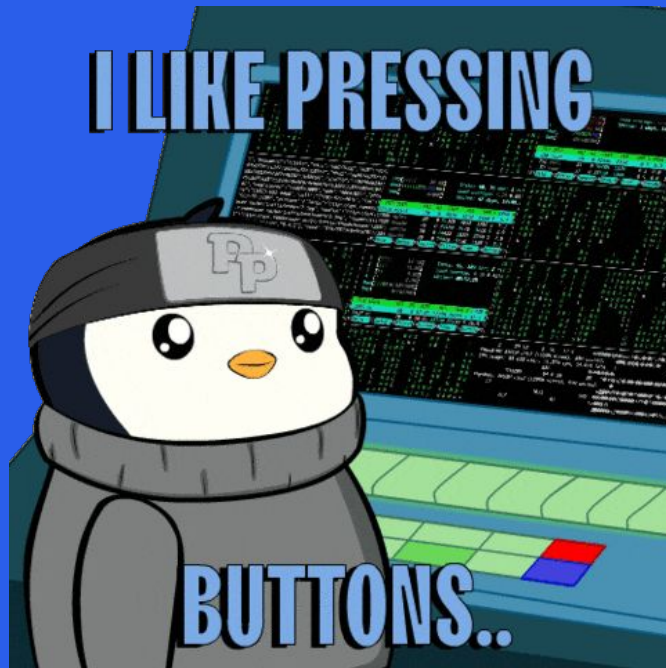


Notebook

Retrieval Agents

- Retrieval Augmented Generation (RAG) systems combine the capabilities of data retrieval and generation models to provide context-aware responses.
- **Agentic RAG** → combines autonomous agents with dynamic knowledge retrieval.
- We could retrieve from:
 - The web
 - Custom knowledge bases using a specific tool
 - We could enhance these systems with more capabilities

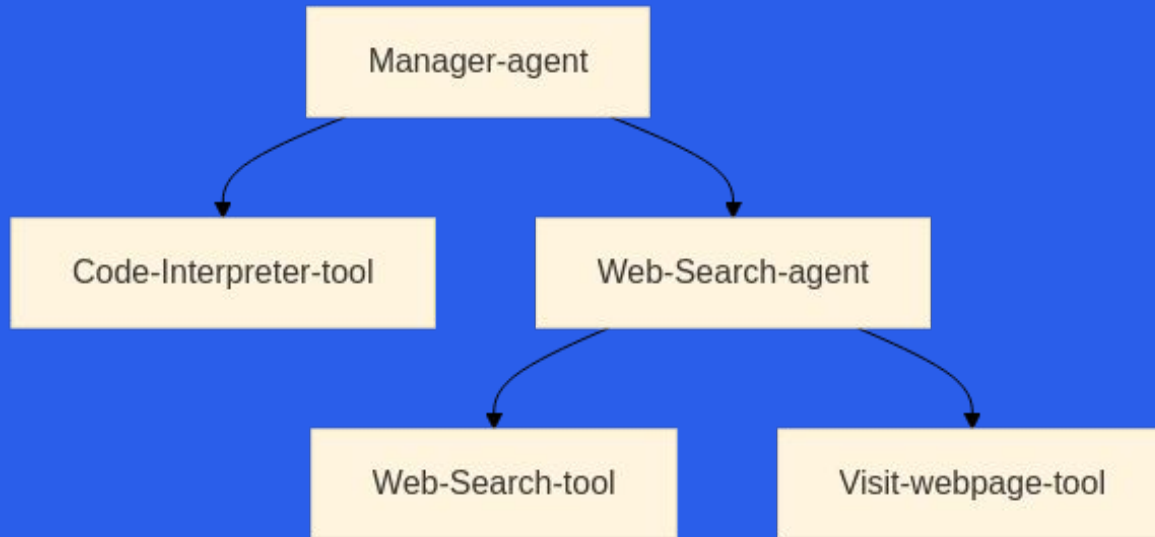
Retrieval Agents



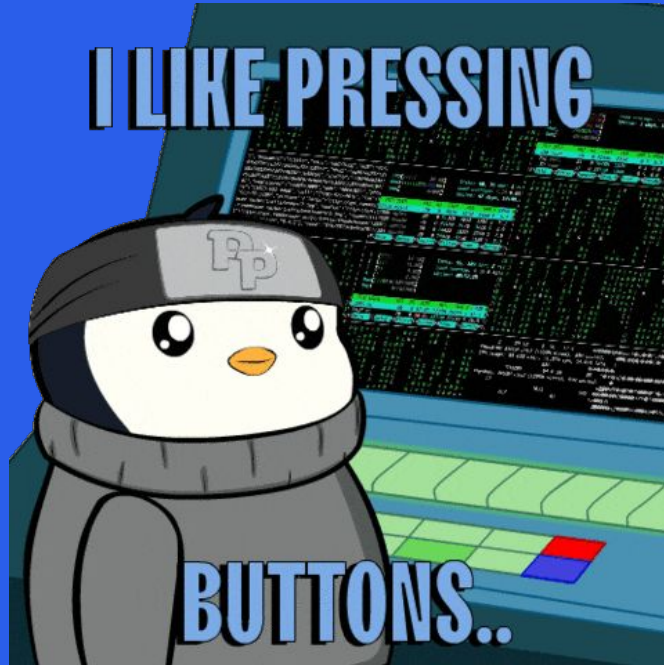
Notebook

Multi-Agent Systems

Enable specialized agents to collaborate on complex tasks.



Multi-Agent Systems



Notebook

If you want to know more



Agents Course

Thanks!

Sergio Paniego Blanco (@sergiopaniego)

Machine Learning Engineer @ Hugging Face

